	UNIVERSITAS ISLAM NEGERI **SYARIF HIDAYATULLAH JAKARTA** **FAKULTAS SAINS DAN TEKNOLOGI** *Jl. Ir. H. Juanda No 95 Ciputat 15412 Indonesia*	**FORM (FR)**	No. Dok.	: FST-AKM-FR-019
			Tgl. Terbit	: 8 Desember 2025
			No. Revisi	: 00
			Hal	: 1/1
SOAL UJIAN AKHIR SEMESTER				

PETUNJUK UMUM UAS (WAJIB DIBACA)

1. Ujian ini berupa satu soal proyek pemrograman yang terdiri dari beberapa bagian (a, b, c, …). Semua bagian wajib dikerjakan.
2. Bahasa pemrograman yang wajib digunakan: Python.
3. Jawaban boleh dalam bentuk:
   - file `.ipynb` (Jupyter Notebook), atau
   - file `.py` (Python script).

4. Dataset sudah disediakan oleh dosen dalam file: `data_aktivitas_50_mahasiswa.py`
   yang berisi variabel: `data_aktivitas = [ ... ]  # list of dict`

5. Batas pengumpulan:
   - Maksimal 21 Desember 2025 pukul 23.59 WIB.
   - Dikirim ke email: `irvanseptiar@gmail.com`
   - Subject email wajib (harus sama persis formatnya):
     `UAS_ALGO&DASPROG2025_NIM_NAMA_KELAS`
     Contoh:
     `UAS_ALGO&DASPROG2025_23012345_BudiSetiawan_A`

6. Di dalam body email, wajib mencantumkan:
   - Link video YouTube atau link Google Drive video
   - Link harus bisa diakses (pengaturan akses sudah benar).
   - Durasi video maksimal 30 menit.

7. Isi video:
   - Menjelaskan struktur data, langkah pemrosesan, dan hasil analisis dari soal di bawah.
   - Menunjukkan eksekusi program (run) dan menjelaskan mengapa hasil yang muncul sudah sesuai.

8. Segala bentuk keterlambatan dengan alasan apapun tidak diterima dan dianggap tidak mengikuti UAS.

SOAL

"Detektif Data: Mencari Mahasiswa Hantu di Aktivitas E-Learning"

Anda adalah seorang data analis yang diminta membangun sebuah program Python untuk menganalisis perilaku belajar mahasiswa dari data aktivitas e-learning yang sudah disediakan. Data aktivitas telah dilampirkan dalam soal ini dengan filename *data_aktivitas_50_mahasiswa_time.json.*

Bagian (a) – Import Data dan Struktur Awal (wajib ada `try/except`)

Dataset UAS disediakan dosen dalam file: `data_aktivitas_50_mahasiswa_time.json` yang dapat diakses pada link berikut: `https://drive.google.com/file/d/1PoQpVVfcU9y5XBswNzaZOSN6Kys41ZVI/view?usp=sharing`

File ini berisi data dalam format JSON yang ketika dibaca akan menjadi variabel **`data_aktivitas`** bertipe **list of dict** dengan struktur seperti yang dijelaskan di awal soal. Pada bagian ini, Anda diminta untuk:

1. Membaca file JSON dan menyimpannya ke variabel **`data_aktivitas`**.
2. Menangani kemungkinan error dengan **`try/except`**.
3. Menampilkan jumlah mahasiswa yang terbaca.

Ketentuan Teknis (WAJIB)

1. Anda **wajib** menggunakan blok `try/except` saat membaca file JSON, dengan ketentuan minimal menangani dua jenis error berikut:
   - `FileNotFoundError` → jika file `data_aktivitas_50_mahasiswa.json` tidak ditemukan.
   - `json.JSONDecodeError` → jika isi file bukan JSON yang valid (misalnya korup/rusak).

2. **Catatan penting:**
   - Tidak diperbolehkan hanya menulis `except:` tanpa menyebut jenis error.
   - Pesan kesalahan yang dicetak (`print`) harus **jelas** menjelaskan masalahnya, bukan hanya teks umum seperti `"error"`.

3. Jika pembacaan file **berhasil**, Anda **wajib**:
   - Memastikan bahwa `data_aktivitas` terisi.
   - Menampilkan jumlah mahasiswa dengan: `print("Jumlah mahasiswa di data_aktivitas:", len(data_aktivitas))`

- Diasumsikan bahwa setiap elemen di `data_aktivitas` merepresentasikan **satu mahasiswa**.

4. Jika pembacaan file **gagal** (terkena `FileNotFoundError` atau `JSONDecodeError`):

    - Program harus dihentikan dengan rapi, misalnya menggunakan `raise SystemExit(...)` atau mekanisme yang setara.

    - **Tidak boleh** melanjutkan ke bagian (b), (c), dan seterusnya dalam keadaan `data_aktivitas` kosong atau tidak valid.

Bagian (b) – Fungsi `flatten_aktivitas(data_aktivitas)`

Buat fungsi:

```
def flatten_aktivitas(data_aktivitas):
    """
    Menerima data_aktivitas (list of dict, sesuai dataset dosen)
       dan mengembalikan DataFrame pandas dengan satu baris per
aktivitas.
    """
    ...
```

Fungsi ini **wajib**:

- Menghasilkan sebuah pandas.DataFrame dengan **minimal kolom** berikut:

    - nim (string)
    - nama (string)
    - kelas (string)
    - tanggal (**tipe datetime**, hasil konversi dari string "YYYY-MM-DD")
    - waktu_mulai (string, format "HH:MM:SS", dibaca dari dataset apa adanya)
    - waktu_selesai (string, format "HH:MM:SS", dibaca dari dataset apa adanya)
    - datetime_mulai (**datetime** hasil gabungan tanggal + waktu_mulai)
    - datetime_selesai (**datetime** hasil gabungan tanggal + waktu_selesai)
    - jenis_aktivitas (string, contoh: "tonton_video", "kerjakan_quiz", "baca_materi", "forum_diskusi")
    - durasi_menit (numerik, int atau float, **dihitung** dari selisih datetime_selesai - datetime_mulai dalam satuan menit)
    - skor (numerik)
    - halaman (numerik)

- Setiap elemen di log_harian dan setiap elemen di list aktivitas di dalamnya **harus di-flatten** sehingga **setiap aktivitas individual menjadi satu baris** di DataFrame.

- **Penanganan data tidak lengkap (WAJIB menggunakan `try/except` di dalam fungsi ini):**

  Di dalam proses membaca dan mengonversi field:

  - Jika `waktu_mulai` atau `waktu_selesai` **tidak ada** / key tidak ditemukan / format waktunya tidak valid:
    - Tangani dengan `try/except`
    - Isi:
      - `datetime_mulai = NaT` (Not a Time) atau `None`
      - `datetime_selesai = NaT` atau `None`
      - `durasi_menit = 0`
  - Jika `skor` tidak ada → isi `skor = 0`.
  - Jika `halaman` tidak ada → isi `halaman = 0`.
  - Jika terjadi `KeyError` atau `TypeError` saat membaca field apa pun, **wajib** ditangani dengan `try/except` dan **tetap** buat baris untuk aktivitas tersebut, dengan nilai default 0 atau NaT untuk field yang bermasalah (sesuai jenisnya).
  - **Tidak boleh** menghapus / skip aktivitas hanya karena datanya tidak lengkap. Semua aktivitas **harus tetap masuk** ke `DataFrame`, dengan pengisian nilai default seperti di atas.
- **Setelah fungsi dipanggil:** `df_aktivitas = flatten_aktivitas(data_aktivitas)`
- Anda **wajib menampilkan**:

```
print(df_aktivitas.head(10))
print("Jumlah baris df_aktivitas:", len(df_aktivitas))
```

Bagian (c) – Membentuk `df_perilaku` per Mahasiswa

Dari `df_aktivitas`, buat `DataFrame` ringkasan per mahasiswa bernama `df_perilaku` dengan **satu baris per NIM**, berisi minimal kolom:

1. `nim`
2. `nama`
3. `kelas`
4. `total_hari_aktif` = jumlah **tanggal unik** di mana mahasiswa tersebut memiliki aktivitas (apa pun jenisnya).
5. `total_aktivitas` = jumlah baris di `df_aktivitas` untuk mahasiswa tersebut.
6. `total_durasi_video` = total `durasi_menit` **hanya** untuk baris dengan `jenis_aktivitas == "tonton_video"`.
7. `jumlah_quiz` = jumlah baris dengan `jenis_aktivitas == "kerjakan_quiz"`.
8. `rata2_skor_quiz`
   - rata-rata `skor` **hanya** untuk baris dengan `jenis_aktivitas == "kerjakan_quiz"` dan `skor > 0`;

    - jika mahasiswa tersebut **tidak punya quiz dengan skor > 0**, isi `NaN` atau `0`, dan jelaskan pilihan Anda di video.

9. `jumlah_jenis_aktivitas_berbeda`
   = jumlah jenis aktivitas unik (`jenis_aktivitas`) yang pernah dilakukan mahasiswa tersebut (misalnya hanya `"tonton_video"` → 1, kalau `"tonton_video"` + `"kerjakan_quiz"` → 2, dst.).

Pembentukan `df_perilaku` boleh menggunakan kombinasi:

- `groupby`
- `agg`
- `apply`
- dan/atau looping manual.

Yang penting: definisi tiap kolom di atas **dipenuhi dengan benar**.

Bagian (d) – Kriteria "POTENSIAL_HANTU" vs "NORMAL" (TIDAK BOLEH ABSTRAK)

Untuk menghindari ambiguitas, gunakan **kriteria tegas berikut** untuk menentukan label:
Seorang mahasiswa diberi label **`"POTENSIAL_HANTU"`** jika **semua syarat berikut terpenuhi**:

1. `total_aktivitas >= 30`
2. `total_durasi_video >= 600` menit
3. `jumlah_quiz <= 1`
4. `jumlah_jenis_aktivitas_berbeda <= 2`

Jika **salah satu saja** syarat di atas tidak terpenuhi, mahasiswa tersebut diberi label **`"NORMAL"`**.
Buat fungsi:

```
def label_hantu(row):
    ...
```

yang:

- menerima satu baris dari `df_perilaku`, dan
- mengembalikan `"POTENSIAL_HANTU"` atau `"NORMAL"` sesuai aturan di atas.

Tambahkan kolom baru `label` ke `df_perilaku`: `df_perilaku["label"] = df_perilaku.apply(label_hantu, axis=1)`

Bagian (e) – Ringkasan per Kelas dalam Bentuk `dict` Nested

Buat sebuah struktur data Python bernama `ringkasan_hantu` dengan format:

```
ringkasan_hantu = {
    "A": {
        "total_mahasiswa": ...,
        "jumlah_potensial_hantu": ...,
        "persen_potensial_hantu": ...  # dalam persen, misal 12.5
    },
    "B": {
        ...
    },
    "C": {
        ...
    }
}
```

Aturan:

1. `total_mahasiswa` = jumlah baris di `df_perilaku` untuk kelas tersebut.
2. `jumlah_potensial_hantu` = jumlah mahasiswa di kelas tersebut dengan `label == "POTENSIAL_HANTU"`.
3. `persen_potensial_hantu` = `jumlah_potensial_hantu / total_mahasiswa * 100`
   Boleh dibulatkan 1–2 angka di belakang koma (jelaskan di video jika dibulatkan).

Pembuatan `ringkasan_hantu` **wajib menggunakan looping** (`for`), misalnya:

```
ringkasan_hantu = {}
for kelas in sorted(df_perilaku["kelas"].unique()):
    …
```

Bagian (f) – Laporan Teks dengan Looping

Gunakan ringkasan_hantu untuk mencetak laporan ke layar dengan format seperti:

```
Kelas A
 Total mahasiswa        : ...
 Jumlah POTENSIAL_HANTU   : ...
 Persentase POTENSIAL_HANTU: ... %

Kelas B
 ...
```

Aturan:

1. Wajib menggunakan **loop for** untuk membaca isi ringkasan_hantu.

2. Format tidak harus persis sama, tetapi informasi tiga baris di atas harus ada untuk setiap kelas.

Bagian (g) – Analisis Singkat (disajikan di kode dan dijelaskan di video)

1. Tampilkan daftar mahasiswa (NIM, nama, kelas) yang berlabel `"POTENSIAL_HANTU"`.
2. Pilih **minimal 1 mahasiswa** dengan label `"POTENSIAL_HANTU"` dan **1 mahasiswa** dengan label `"NORMAL"` dari kelas yang sama.
3. Di dalam **kode** (boleh pakai `print` multi-baris atau komentar panjang), tuliskan penjelasan singkat:
   - Kenapa mahasiswa pertama bisa berlabel `"POTENSIAL_HANTU"` jika dilihat dari:
     - `total_durasi_video`
     - `jumlah_quiz`
     - `jumlah_jenis_aktivitas_berbeda`
   - Kenapa mahasiswa kedua dianggap `"NORMAL"`.
4. Di video, Anda **wajib menjelaskan** analisis tersebut seperti sedang "menceritakan kasus":
   - "Mahasiswa ini kelihatannya banyak menonton video, tapi hampir tidak pernah quiz, sehingga memenuhi semua syarat POTENSIAL_HANTU…"

Bagian (h) - Sequence Pattern Mining: Apakah Mahasiswa Mengikuti Urutan Ideal?

Desain *"ideal sequence"* aktivitas harian:

`tonton_video` → `baca_materi` → `kerjakan_quiz` → `forum_diskusi`

Tugas:

1. Buat fungsi `extract_sequence(nim, df_aktivitas)` → hasilkan list aktivitas per hari
   Implementasi scoring:
   - Jika urutan mengikuti ideal = skor 1
   - Jika urutan terbalik = skor 0.5
   - Jika tidak mengandung semua = skor 0
2. Hitung `rata2_urut_ideal` per mahasiswa

Tambahkan ke `df_perilaku` dan analisis apakah mahasiswa dengan skor urutan rendah cenderung POTENSIAL_HANTU?

Bagian (i) - Windowed Session Analysis: Efektivitas Sesi Belajar

Konsep: Satu "sesi belajar" didefinisikan sebagai rangkaian aktivitas dengan jeda waktu antar aktivitas **≤ 30 menit**.

Tugas:

1. Buat fungsi: `def segment_sessions(df_aktivitas_per_mahasiswa): …`
    a. Untuk setiap mahasiswa, bagi aktivitasnya per tanggal dan sort berdasarkan waktu.
    b. Bentuk sesi belajar dengan aturan: jika jeda waktu antar aktivitas > 30 menit → sesi baru.
    c. Berikan ID sesi unik (`session_id`) ke setiap aktivitas.

2. Hitung statistik per mahasiswa:
   `rata2_durasi_sesi`: rata-rata total durasi_menit per sesi
    - `rata2_aktivitas_per_sesi`: rata-rata jumlah aktivitas per sesi
    - `jumlah_sesi`
3. Tambahkan ke `df_perilaku`

Analisis:

- Siapa mahasiswa dengan sesi belajar paling panjang atau paling padat?
- Apakah mahasiswa "POTENSIAL_HANTU" punya sesi belajar yang mencurigakan (misalnya durasi sesi sangat lama tapi hanya 1 aktivitas)?
- Gunakan visualisasi scatterplot: rata2_durasi_sesi vs rata2_aktivitas_per_sesi per label